

La pile graphique Linux/Wayland

Quelle pièce fait quoi, pourquoi, et comment elles communiquent

Auteur : **noise**

De quoi parle ce document. Sous Linux, faire tourner un bureau graphique met en jeu plusieurs programmes empilés : ceux qui parlent au matériel, celui qui gère les fenêtres, ceux qui rendent des services (son, notifications), et ceux qui dessinent la barre et les widgets. Ce document explique **quelle pièce fait quoi, pourquoi elle existe, et comment elle communique avec les autres.**

La lecture se fait en cascade, du plus bas (matériel) au plus haut (ce que tu cliques). Chaque couche s'appuie sur celle du dessous.

0 Matériel + noyau

GPU, écran, clavier, souris. Seul le **noyau Linux** les touche, via :

- [DRM/KMS](#) → pilote les cartes graphiques et les sorties vidéo (résolution, rafraîchissement, envoi de l'image à l'écran).
- [libinput](#) (sur [evdev](#)) → lit les entrées physiques (clavier, souris, touchpad).

Tout ce qui est au-dessus devra passer par un intermédiaire pour atteindre le matériel.

1 Briques bas niveau — [wlroots](#) / [smithay](#) / [aquamarine](#)

Des bibliothèques qui font la plomberie entre le compositeur et le noyau : parler à DRM/KMS et libinput, allouer les buffers GPU, composer l'image, gérer le multi-écran. Elles fournissent aussi des implémentations prêtes des extensions de protocole courantes.

- [wlroots](#) (C) → utilisée par Sway
- [smithay](#) (Rust) → utilisée par Niri
- [aquamarine](#) → utilisée par Hyprland

Ce ne sont pas des compositeurs : ce sont les fondations sur lesquelles un compositeur se construit. Elles ne décident rien sur le placement des fenêtres.

2 Protocole — Wayland

Définit **comment les applications et le compositeur se parlent**. Ce n'est pas un programme mais une spécification.

- **Orienté objet** : le protocole définit des interfaces (types d'objets) avec des *requêtes* (client → compositeur) et des *événements* (compositeur → client). Ex. : [wl_surface](#) (une surface), [wl_pointer](#)/[wl_keyboard](#) (entrées), [xdg_toplevel](#) (une fenêtre). Les messages transitent sur un **socket Unix**.
- **Asynchrone** : le client envoie ses requêtes sans bloquer en attente d'une réponse. (X11, l'ancêtre, imposait des allers-retours synchrones, plus lents.)
- **Découverte via le registry** : le client part de [wl_display](#), en obtient le [wl_registry](#) (le catalogue), où le compositeur annonce ses interfaces globales. Le client se **lie** (bind) à

celles qu'il veut, créant les objets dont il a besoin.

- **Les pixels ne passent pas par le socket** : le socket ne porte que les messages de contrôle. Les pixels sont écrits dans une mémoire partagée (shm) ou un buffer GPU (dma-buf) ; le client envoie juste un *handle*, le compositeur y lit les pixels sans copie.
- **Frontière** : Wayland ne régit que la conversation appli → compositeur. Le dialogue avec le matériel reste au noyau (couche 0), via les briques (couche 1).

Extensions (le protocole de base est minimal, négociées via le registry) :

- `xdg-shell` : fenêtres applicatives normales
- `wlr-layer-shell` : surfaces ancrées aux bords, hors du flux des fenêtres → barres
- `idle-notify` : le compositeur signale l'inactivité → idle daemon

3 Compositeur — Sway / Hyprland / Niri

Le programme qui **implémente** Wayland (couche 2) en s'appuyant sur une brique bas niveau (couche 1). Sous Wayland il cumule trois rôles séparés sous X11 :

1. **Serveur d'affichage** → pousse l'image au GPU
2. **Compositeur** → assemble les surfaces des clients en une image
3. **Window manager** → placement, workspaces, comportement des fenêtres

On en installe un seul. Différence principale = le rôle n°3 :

Compositeur	Langage	Base	Fenêtrage
Sway	C	wlroots	Tiling en arbre (i3-like)
Hyprland	C++	aquamarine	Tiling dynamique + effets
Niri	Rust	smithay	Tiling scrollable (ruban horizontal)

« Basé sur wlroots » = délègue GPU/entrées/rendu à la brique, n'écrit que sa logique de fenêtrage.

Un compositeur nu ne fournit aucun élément de bureau (ni barre, ni notif, ni son). D'où les couches suivantes.

4 Daemons — services en arrière-plan

Le compositeur affiche des surfaces mais ne rend aucun service. Ces daemons comblent le manque. Ils ne communiquent pas via Wayland (qui ne sert qu'à afficher) mais principalement via **D-Bus**.

D-Bus : système de communication inter-processus, distinct de Wayland. Un processus s'enregistre sous un nom de service (ex. [org.freedesktop.Notifications](#)) et expose des méthodes que d'autres appellent.

Service	Qui l'appelle	Canal demande	Canal affichage
Notifications	applis	D-Bus	Wayland (layer-shell)
PipeWire (audio)	applis, widget barre	API PipeWire / D-Bus	— (parle à ALSA)
polkit	applis privilégiées	D-Bus	agent → Wayland
xdg portal	applis sandboxées	D-Bus	backend → Wayland
idle daemon	le compositeur notifie	Wayland idle-notify	Wayland

Exemple notification :

```

Firefox
| D-Bus: Notify()
▼
daemon (mako/dunst)
| Wayland: layer-shell
▼
compositeur → écran

```

Un daemon a souvent une double casquette : serveur D-Bus (il reçoit la demande) et client Wayland (quand il doit dessiner). Exception : l'idle daemon passe par l'extension Wayland idle-notify, car l'info d'inactivité vient du compositeur.

Principe : Wayland = affichage + entrées ; tout autre service passe par D-Bus.

5 Shell — Quickshell / Waybar / EWW

La couche visuelle que tu utilises pour piloter le système (« shell » = coquille du bureau, pas le terminal). Recouvre : barre de statut, launcher, notifications, systray, wallpaper, lock.

- **Quickshell** → en Qt/QML : tu décris l'interface en QML, le moteur (C++/Qt) fait le lien avec le compositeur et le système. Permet une shell complète.
- **Waybar** → la barre seule. **EWW** → widgets.

Il s'affiche en se déclarant au compositeur via `wlr-layer-shell` (couche 2) pour se coller à un bord d'écran. Sans cette extension, pas de barre positionnable.

6 WM vs DE

- **WM / compositeur** : gère seulement les fenêtres (couche 3 nue).
- **DE** : pack intégré = WM + shell + daemons + applis + thème, d'un bloc.
 - GNOME → Mutter + GNOME Shell
 - KDE → KWin + Plasma Shell
 - XFCE → Xfwm
- Le DE **contient** un WM : ce n'est pas l'un ou l'autre.
- Démarche ricing : pas de DE, on assemble à la carte — compositeur nu (3) + daemons choisis (4) + shell Quickshell (5).

Exemple KDE Plasma dans la cascade

- Couche 3 : **KWin** (compositeur/WM, backend propre, pas wlroots)

- Couche 4 : daemons KDE (notifs, audio, polkit, portal [xdg-desktop-portal-kde](#))
- Couche 5 : **Plasma Shell** (barre, launcher, systray, wallpaper) — l'équivalent de Quick-shell

Même pile qu'un setup à la carte, mais livrée d'un bloc et intégrée.

La cascade

```
6. WM vs DE      à la carte, ou DE tout intégré
5. SHELL         Quickshell/Waybar – barre, widgets (wlr-layer-shell)
4. DAEMONS       notifs, PipeWire, polkit, portal (via D-Bus)
3. COMPOSITEUR   Sway/Hyprland/Niri = affichage + compositeur + WM
2. PROTOCOLE     Wayland (objets, async, registry, extensions)
1. BRIQUES       wlroots/smithay/aquamarine – plomberie GPU/entrées
0. NOYAU         DRM/KMS, libinput – GPU, écran, clavier, souris
```

Seul le compositeur (3) traverse toute la pile : il parle au noyau (0) via les briques (1), implémente le protocole (2), et sert d'ancrage aux daemons (4) et au shell (5).